

TP5 - Virtualisation, projet noté

Module : R4.A.08 - Outils de la virtualisation - TP5 : Projet final d'architecture Docker Durée : 3h00

Contexte et Objectifs

Ce projet a pour but d'évaluer votre capacité à concevoir, configurer et déployer une infrastructure d'entreprise "complète" et "sécurisée" via Docker Compose.

Vous travaillerez en **binôme** (ou **trinôme** si effectif impair).

L'ensemble de votre infrastructure doit être déployé par une **unique commande** et respecter des principes stricts **d'isolation réseau (plusieurs réseaux)** et de **haute disponibilité (la bdd utilisée sera dupliquée)**.

Cahier des charges de l'infrastructure

Votre fichier `docker-compose.yml` doit instancier l'architecture suivante.

Attention : le respect strict des noms de conteneurs et de réseaux est obligatoire car l'évaluation sera automatisée.

1. Réseaux (**Isolation**) Vous devez créer 3 réseaux custom de type `bridge` :
 - o `net_public` : Zone exposée.
 - o `net_db` : Zone des bases de données applicatives.
 - o `net_auth` : Zone dédiée à l'authentification.
2. Reverse Proxy (**Nginx**), (cf annexe 1).
 - o Nom de conteneur : `proxy_nginx`
 - o Rôle : Point d'entrée unique de la stack. Expose les ports `80` (HTTP) et `443` (HTTPS) sur la machine hôte. Il **redirige** le trafic **vers le service web**.
 - o Réseau : `net_public` uniquement.
3. Serveur Web Applicatif
 - o Nom de conteneur : `web_app`
 - o Rôle : Serveur Apache/PHP ou NodeJS (créé via un `Dockerfile` personnalisé avec les extensions nécessaires). Ne doit exposer aucun port directement sur la machine hôte.
 - o Réseaux : `net_public` et `net_db`.
4. Bases de données (**Haute Disponibilité**)
 - o Noms de conteneurs : `mysql_master` et `mysql_slave` (images `mysql:9.2`).
 - o Rôle : Le master reçoit les écritures, le slave réplique les données. Les données et les fichiers de configuration personnalisés (`master.cnf`, `slave.cnf`) doivent être montés via des volumes. Les mots de passe doivent provenir d'un fichier `.env`.
 - o Réseau : `net_db` uniquement.
 - o Note : La configuration de la réplication se fera via des commandes `docker exec` une fois les conteneurs lancés.

5. Authentification (**Keycloak**, cf annexe 2)

- Conteneurs : **keycloak_auth** (image **quay.io/keycloak/keycloak:latest**) et sa base de données **postgres_keycloak** (image **postgres:15**). **Attention** : pour Keycloak, il faut impérativement surcharger la commande de démarrage du conteneur avec **start-dev** pour qu'il puisse fonctionner en environnement local.
- Rôle : Gestion des identités. Keycloak doit attendre (**orchestration, annexe 3**) que Postgres soit démarré pour se lancer.
- Réseaux : **net_public** et **net_auth** pour Keycloak. **net_auth** uniquement pour Postgres.

6. Serveur de Messagerie

- Conteneur : **mail_test** (image **axllent/mailpit**).
- Rôle : Intercepteur SMTP pour tester l'envoi de mails depuis le serveur web. Expose le port **8025** pour consulter l'interface web.
- Réseau : **net_public**.

Livrables, créer un dépôt privé sur GitLab, donnez l'accès à votre enseignant (-5 sinon), renseigner le lien dans le fichier [ici](#)). On retrouvera sur votre dépôt :

1. Les fichiers de configuration (**.env**, **docker-compose.yml**, **Dockerfile**, ***.cnf**, **nginx.conf**).
2. Le fichier **.gitignore**.
3. Un fichier **README.md** (format Markdown strict) contenant :
 - Un schéma ou une explication des flux réseaux.
 - La commande de déploiement.
 - Le mode opératoire exact (commandes exécutées) pour lier le master et le slave MySQL.

ANNEXES : Fiche Ressources

Annexe 1 : Reverse Proxy

Un *Reverse Proxy* est un serveur placé devant vos serveurs web. Il écoute les requêtes des clients (sur le port 80 ou 443) et les relaie vers les conteneurs internes. *Exemple de bloc server Nginx minimal (nginx.conf)* :

```
Nginx
server {
    listen 80;
    server_name localhost;
    location / {
        proxy_pass http://web_app:80; # Nom du conteneur cible
        proxy_set_header Host $host;
    }
}
```

Annexe 2 : Qu'est-ce que Keycloak (Gestion des identités) ?

Définition : Keycloak est une solution Open Source très populaire de gestion des identités et des accès (IAM) développée par Red Hat. Il permet de mettre en place du Single Sign-On (SSO), c'est-à-dire une authentification unique.

Son rôle dans votre infrastructure :

Dans une architecture moderne, le serveur Web applicatif ne gère plus lui-même les mots de passe des utilisateurs (ce qui représente un risque de sécurité). Il délègue cette tâche à un tiers de confiance : **Keycloak**.

Lorsqu'un utilisateur veut accéder à l'application Web, celle-ci le redirige vers la page de connexion sécurisée de Keycloak.

Keycloak vérifie les identifiants et renvoie un "jeton" d'accès (Token) à l'application Web.

L'application Web **laisse** alors l'utilisateur entrer.

Pourquoi a-t-il besoin de PostgreSQL ?

Keycloak est un moteur applicatif (développé en Java). Pour fonctionner, il a besoin de sauvegarder sa propre configuration, les "Royaumes" (Realms), les clients, ainsi que les utilisateurs et leurs mots de passe chiffrés. Il requiert donc sa propre base de données relationnelle dédiée (ici, Postgres), totalement isolée de la base de données métier (MySQL).

Note technique pour le TP : Pensez à consulter la documentation officielle de l'image Docker Keycloak sur Internet pour trouver quelles variables d'environnement (par exemple : KC_DB, KC_DB_URL, etc.) vous devez définir dans votre fichier Compose pour réussir à le lier à votre conteneur Postgres.

Annexe 3 : Ordonnancement (Orchestration)

Certains services plantent s'ils démarrent avant leurs dépendances (ex: Keycloak sans sa base Postgres). Utilisez la directive `depends_on` dans votre YAML (comme vu en cours) :

YAML

services:

mon_app:

image: mon_appli:latest

depends_on:

- ma_bdd # Attend ce du conteneur